

领域驱动设计规范

技术规范文件

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

目录

- 1. 概述
- 2. 战略设计
 - 2.1 统一语言
 - 2.2 限界上下文
 - 2.3 上下文映射
- 3. 战术设计
 - 3.1 聚合与聚合根
 - 3.2 聚合设计原则
 - 3.3 仓储模式
 - 3.4 领域事件
- 4. 代码结构规范（推荐包结构）
- 5. 与现有系统集成

领域驱动设计规范

1. 概述

本文档定义保险技术平台中应用领域驱动设计(DDD)的实践规范，包括战略设计和战术设计两个层次的指导原则。参考 Eric Evans 原著及蚂蚁集团、美团的 DDD 实践经验。

2. 战略设计

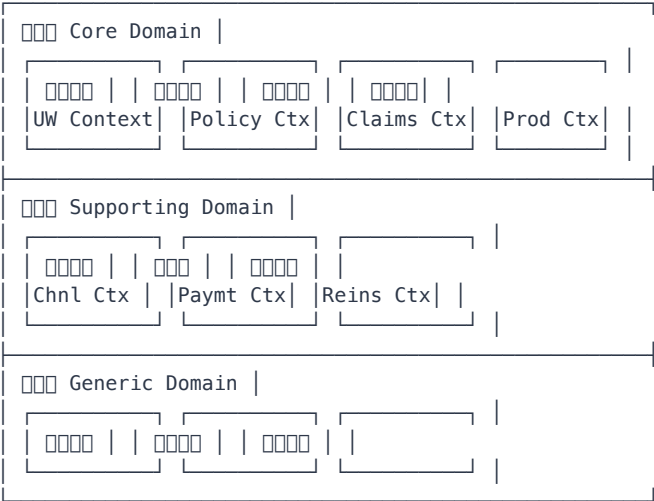
2.1 统一语言

所有项目成员使用统一语言沟通，词汇表存储在项目 Wiki 中。常见保险领域词汇示例：

术语	含义	英文
投保单	客户提交的投保申请（未生效）	Application
保单	已生效的保险合同	Policy
保全	保单生效后的变更操作	Endorsement / Servicing
核保	风险评估决策过程	Underwriting
理赔	保险事故处理过程	Claims
保费	投保人缴纳的费用	Premium
保额	保险金额	Sum Insured
免赔额	自付部分	Deductible

2.2 限界上下文

保险核心领域上下文划分：



2.3 上下文映射

关系	模式	示例
合作关系	Partnership	承保上下文 ↔ 收付上下文
共享内核	Shared Kernel	保单 ↔ 客户共享客户模型
客户-供应商	Customer-Supplier	产品上下文 → 承保上下文
分离方式	Separate Ways	承保 vs 理赔，各自独立发版

关系	模式	示例
防腐层	Anti-Corruption Layer	对接核心系统遗留系统时引入

3. 战术设计

3.1 聚合与聚合根

上下文	聚合根	实体	值对象
承保	Application	ApplicationItem, UnderwritingRecord	ApplicantInfo, RiskEvaluation
保单	Policy	PolicyInsured, PolicyBeneficiary, PolicyCoverage	PremiumAmount, CoveragePeriod
理赔	Claim	ClaimItem, AssessmentResult, PaymentRecord	DamageAssessment, SettlementAmount
产品	Product	ProductClause, ProductCoverage, RateTable	RateFactor, CommissionRule
收付	PaymentOrder	PaymentRecord, ReconciliationEntry	PaymentMethod, BankAccount

3.2 聚合设计原则

1. 聚合内保证最终一致性，聚合间通过事件驱动
2. 跨聚合引用使用 ID，不直接持有对象引用
3. 事务边界不超出单个聚合
4. 一次请求最多修改一个聚合
5. 聚合大小控制在 1 次 DB 事务可覆盖的范围

3.3 仓储模式

每个聚合根对应一个 Repository 接口：

```
public interface PolicyRepository {
    Policy findById(PolicyId id);
    void save(Policy policy);
    void delete(PolicyId id);
    // 查询
    Page<Policy> findByInsured(InsuredId insuredId, Pageable pageable);
}
```

3.4 领域事件

```
// 领域事件
public class PolicyIssuedEvent implements DomainEvent {
    private final PolicyId policyId;
    private final Instant occurredOn;
    // ...
}

// 聚合根
public class Policy extends AggregateRoot {
    public void issue() {
        // ... 发布
        this.addDomainEvent(new PolicyIssuedEvent(this.id, Instant.now()));
    }
}
```

关键领域事件	触发场景	订阅者
PolicyIssuedEvent	保单签发	渠道(佣金)、再保(自动分保)、收付(应收)
PremiumPaidEvent	保费到账	保单(状态变更)、续期(更新记录)
ClaimSettledEvent	理赔结案	收付(支付)、再保(摊回)
PolicyLapsedEvent	保单失效	续期(停止催缴)、渠道(佣金调整)

4. 代码结构规范（推荐包结构）

```

com.company.insurance.underwriting
├── application # 应用
│   └── UnderwritingAppService.java
├── domain # 领域
│   ├── aggregate
│   │   └── Application.java
│   ├── entity
│   ├── vo
│   ├── event
│   ├── repository
│   ├── service # 应用服务
│   └── spec # 领域规格
├── infrastructure # 基础设施
│   ├── persistence
│   ├── mq
│   └── client
├── interfaces # 接口
├── rest
├── rpc
└── job
  
```

5. 与现有系统集成

遗留系统采用防腐层(ACL)模式适配：

DDD(DDD) → Anti-Corruption Layer → DDD

ACL 职责：

- 翻译旧系统的数据结构到新领域模型
- 提供新服务期望的接口契约
- 隔离旧系统的变更影响